

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Российский экономический университет им. Г.В. Плеханова»

Сопроводительная записка

по дисциплине «Поддержка и тестирование программных модулей»

на тему: Практическая работа №2

Специальность 09.02.04 «Информационные системы и программирование»

Исполнитель

_____/Д.Ю. Чащин/

Группа ИПо-22

«3» Октября 2026 г.

Пермь, 2026

Оглавление

1. Введение	2
2. Постановка задачи	3
3. Создание решения в Microsoft Visual Studio	3
4. Создание библиотеки классов	4
4.1. Добавление проекта	4
4.2. Реализация класса Calculator	4
5. Создание WPF-приложения	4
5.1. Добавление проекта	5
5.2. Подключение ссылки на библиотеку классов	5
5.3. Разработка интерфейса (MainWindow.xaml)	5
5.4. Реализация логики интерфейса (MainWindow.xaml.cs)	5
6. Создание проекта модульных тестов	7
6.1. Добавление проекта	7
6.2. Написание тестов	7
6.3. Запуск тестов	8
7. Запуск и проверка работы приложения	9
8. Заключение	10

1. Введение

Целью данной работы является разработка простого приложения «**Калькулятор**», реализующего базовые арифметические операции: сложение, вычитание, умножение, деление и возведение в степень. Работа выполняется на языке **C#** с использованием технологии **WPF** (Windows Presentation Foundation) и среды разработки **Microsoft Visual Studio 2022**.

В соответствии с требованиями задания основная логика приложения вынесена в отдельную **библиотеку классов** (CalculatorIvanov.Core), а пользовательский интерфейс реализован в виде **WPF-приложения** (CalculatorIvanov.WPF). Дополнительно разработан проект **модульных тестов** (CalculatorIvanov.Tests) на основе библиотеки **xUnit**, что позволяет проверить корректность работы всех арифметических операций.

Такое разделение обеспечивает:

- независимое тестирование логики;
- повторное использование библиотеки в других проектах;
- чистоту архитектуры «интерфейс ↔ логика».

Задачи работы:

1. Изучить создание проектов в Microsoft Visual Studio.
2. Реализовать библиотеку классов с пятью арифметическими операциями.
3. Разработать WPF-интерфейс калькулятора.
4. Написать модульные тесты для всех методов.
5. Проверить работу приложения и тестов.

2. Постановка задачи

Разрабатываемое приложение должно:

- выполнять арифметические операции $+$, $-$, $*$, $/$, $^$ (возведение в степень);
- корректно обрабатывать деление на ноль (выводить сообщение об ошибке);
- иметь графический интерфейс с кнопками 0–9, знаками операций, кнопками $=$, C (сброс), $\leftarrow$ (удаление последнего символа), $\pm$ (смена знака), $.$ (десятичная точка);

- содержать в названии проекта фамилию студента в английской транскрипции;
- основную логику реализовывать в отдельной библиотеке классов;
- иметь модульные тесты, покрывающие все методы библиотеки.

3. Создание решения в Microsoft Visual Studio

Шаги:

1. Запустить Microsoft Visual Studio 2022.
2. В стартовом окне выбрать «Создание проекта».
3. В списке шаблонов найти «Пустое решение».
4. Указать имя решения: CalculatorChashchin и папку размещения.
5. Нажать «Создать».

4. Создание библиотеки классов

4.1. Добавление проекта

1. Правой кнопкой по решению → **Добавить** → **Новый проект...**
2. Выбрать шаблон «Библиотека классов (Майкрософт)».
3. Имя проекта: CalculatorChashchin
4. Платформа: .NET 10.0.
5. Нажать «Создать».

4.2. Реализация класса Calculator

Удалить автоматически созданный файл Class1.cs и добавить новый класс Calculator.cs со следующим содержимым:

```
using System;

namespace CalculatorChashchin
{
    Ссылка: 4
    public class Calculator
    {
        Ссылка: 2
        public double Add(double a, double b) => a + b;
        Ссылка: 2
        public double Subtract(double a, double b) => a - b;
        Ссылка: 2
        public double Multiply(double a, double b) => a * b;
        Ссылка: 3
        public double Divide(double a, double b)
        {
            if (b == 0)
                throw new DivideByZeroException("Деление на ноль недопустимо.");
            return a / b;
        }
        Ссылка: 2
        public double Power(double baseValue, double exponent)
        {
            return Math.Pow(baseValue, exponent);
        }
        Ссылка: 3
        public double Calculate(double a, double b, string op)
        {
            return op switch
            {
                "+" => Add(a, b),
                "-" => Subtract(a, b),
                "*" => Multiply(a, b),
                "/" => Divide(a, b),
                "^" => Power(a, b),
                _ => throw new ArgumentException($"Неизвестный оператор: {op}")
            };
        }
    }
}
```

5. Создание WPF-приложения

5.1. Добавление проекта

1. Правой кнопкой по решению → **Добавить** → **Новый проект...**
2. Шаблон «**Приложение WPF**» (WPF Application).
3. Имя проекта: CalculatorChashchin.WPF, платформа .NET 10.0.
4. Нажать «**Создать**».

5.2. Подключение ссылки на библиотеку классов

1. Правой кнопкой по CalculatorChashchin.WPF → **Добавить** → **Ссылка на проект...**
2. Отметить CalculatorChashchin.Core → **ОК**.

5.3. Разработка интерфейса (MainWindow.xaml)

Был написан код для: MainWindow.xaml:

```
<Window x:Class="CalculatorChashchin.WPF.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Калькулятор" Height="420" Width="320"
        WindowStartupLocation="CenterScreen"
        ResizeMode="CanMinimize">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="60"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <TextBox x:Name="Display" Grid.Row="0"
                FontSize="28" TextAlignment="Right"
                VerticalContentAlignment="Center"
                IsReadOnly="True" Text="0"/>
        <UniformGrid Grid.Row="1" Rows="5" Columns="4" Margin="0,10,0,0">
            <Button Content="C" Click="Clear_Click" FontSize="18"/>
            <Button Content="^" Click="Operator_Click" FontSize="18" Tag="^"/>
            <Button Content="←" Click="Backspace_Click" FontSize="18"/>
            <Button Content="/" Click="Operator_Click" FontSize="18" Tag="/" />
            <Button Content="7" Click="Digit_Click" FontSize="18"/>
            <Button Content="8" Click="Digit_Click" FontSize="18"/>
            <Button Content="9" Click="Digit_Click" FontSize="18"/>
            <Button Content="*" Click="Operator_Click" FontSize="18" Tag="*" />
            <Button Content="4" Click="Digit_Click" FontSize="18"/>
            <Button Content="5" Click="Digit_Click" FontSize="18"/>
            <Button Content="6" Click="Digit_Click" FontSize="18"/>
            <Button Content="-" Click="Operator_Click" FontSize="18" Tag="-" />
            <Button Content="1" Click="Digit_Click" FontSize="18"/>
            <Button Content="2" Click="Digit_Click" FontSize="18"/>
            <Button Content="3" Click="Digit_Click" FontSize="18"/>
            <Button Content="+" Click="Operator_Click" FontSize="18" Tag="+" />
            <Button Content="±" Click="Sign_Click" FontSize="18"/>
            <Button Content="0" Click="Digit_Click" FontSize="18"/>
            <Button Content="." Click="Dot_Click" FontSize="18"/>
            <Button Content="=" Click="Equals_Click" FontSize="18"/>
        </UniformGrid>
    </Grid>
</Window>
```

5.4. Реализация логики интерфейса (MainWindow.xaml.cs)

```
namespace CalculatorChashchin.WPF
{
    Ссылка 2
    public partial class MainWindow : Window
    {
        Ссылка 1
        private readonly Calculator _calculator = new Calculator();

        Ссылка 4
        private double _firstOperand;
        Ссылка 6
        private string _pendingOperator;
        Ссылка 10
        private bool _isNewEntry = true;

        Ссылка 0
        public MainWindow()
        {
            InitializeComponent();
        }

        Ссылка 10
        private void Digit_Click(object sender, RoutedEventArgs e)
        {
            var digit = ((Button)sender).Content.ToString();
            if (_isNewEntry || Display.Text == "0")
            {
                Display.Text = digit;
                _isNewEntry = false;
            }
            else
            {
                Display.Text += digit;
            }
        }

        Ссылка 1
        private void Dot_Click(object sender, RoutedEventArgs e)
        {
            if (_isNewEntry)
            {
                Display.Text = "0.";
                _isNewEntry = false;
            }
            else if (!Display.Text.Contains(','))
            {
                Display.Text += ",";
            }
        }
    }
}
```

Был написан код:

```

private void Sign_Click(object sender, RoutedEventArgs e)
{
    if (double.TryParse(Display.Text, out var value))
    {
        value = -value;
        Display.Text = value.ToString(CultureInfo.CurrentCulture);
    }
}

Ссылка 5
private void Operator_Click(object sender, RoutedEventArgs e)
{
    var op = ((Button)sender).Tag.ToString();

    if (!double.TryParse(Display.Text, out var current))
        return;

    if (_pendingOperator != null && !_isNewEntry)
    {
        TryCompute(current, out var result);
        Display.Text = result.ToString(CultureInfo.CurrentCulture);
        _firstOperand = result;
    }
    else
    {
        _firstOperand = current;
    }

    _pendingOperator = op;
    _isNewEntry = true;
}

Ссылка 1
private void Equals_Click(object sender, RoutedEventArgs e)
{
    if (_pendingOperator == null) return;
    if (!double.TryParse(Display.Text, out var second)) return;

    TryCompute(second, out var result);
    Display.Text = result.ToString(CultureInfo.CurrentCulture);
    _pendingOperator = null;
    _isNewEntry = true;
}

Ссылка 2
private void TryCompute(double second, out double result)
{
    result = 0;
    try
    {
        result = _calculator.Calculate(_firstOperand, second, _pendingOperator);
    }
    catch (DivideByZeroException ex)
    {
        MessageBox.Show(ex.Message, "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
        ClearAll();
        result = 0;
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
        result = 0;
    }
}

Ссылка 1
private void Clear_Click(object sender, RoutedEventArgs e) => ClearAll();
Ссылка 2
private void ClearAll()
{
    Display.Text = "0";
    _firstOperand = 0;
    _pendingOperator = null;
    _isNewEntry = true;
}

Ссылка 1
private void Backspace_Click(object sender, RoutedEventArgs e)
{
    if (_isNewEntry || Display.Text.Length <= 1)
    {
        Display.Text = "0";
        _isNewEntry = true;
        return;
    }
    Display.Text = Display.Text.Substring(0, Display.Text.Length - 1);
}
}

```

6. Создание проекта модульных тестов

6.1. Добавление проекта

1. Правой кнопкой по решению → **Добавить** → **Новый проект...**
2. Шаблон «**Проект тестов xUnit**» (xUnit Test Project).
3. Имя: CalculatorChashchin.Tests, платформа .NET 10.0.

6.2. Написание тестов

Удалить UnitTest1.cs, создать CalculatorTests.cs:

```
using CalculatorChashchin;
using CalculatorChashchin;
using System;
using Xunit;

namespace CalculatorChashchin.Tests
{
    Ссылка 0
    public class CalculatorTests
    {
        Ссылка 8
        private readonly Calculator _calc = new Calculator();

        [Theory]
        [InlineData(2, 3, 5)]
        [InlineData(-2, -3, -5)]
        [InlineData(0, 0, 0)]
        [InlineData(2.5, 2.5, 5)]
        Ссылка 0
        public void Add_ReturnsExpected(double a, double b, double expected)
        {
            Assert.Equal(expected, _calc.Add(a, b), 10);
        }

        [Theory]
        [InlineData(5, 3, 2)]
        [InlineData(3, 5, -2)]
        [InlineData(0, 0, 0)]
        Ссылка 0
        public void Subtract_ReturnsExpected(double a, double b, double expected)
        {
            Assert.Equal(expected, _calc.Subtract(a, b), 10);
        }

        [Theory]
        [InlineData(3, 4, 12)]
        [InlineData(-3, 4, -12)]
        [InlineData(0, 100, 0)]
        Ссылка 0
        public void Multiply_ReturnsExpected(double a, double b, double expected)
        {
            Assert.Equal(expected, _calc.Multiply(a, b), 10);
        }

        [Theory]
        [InlineData(10, 2, 5)]
        [InlineData(1, 4, 0.25)]
        [InlineData(-10, 2, -5)]
        public void Divide_ReturnsExpected(double a, double b, double expected)
        {
            Assert.Equal(expected, _calc.Divide(a, b), 10);
        }

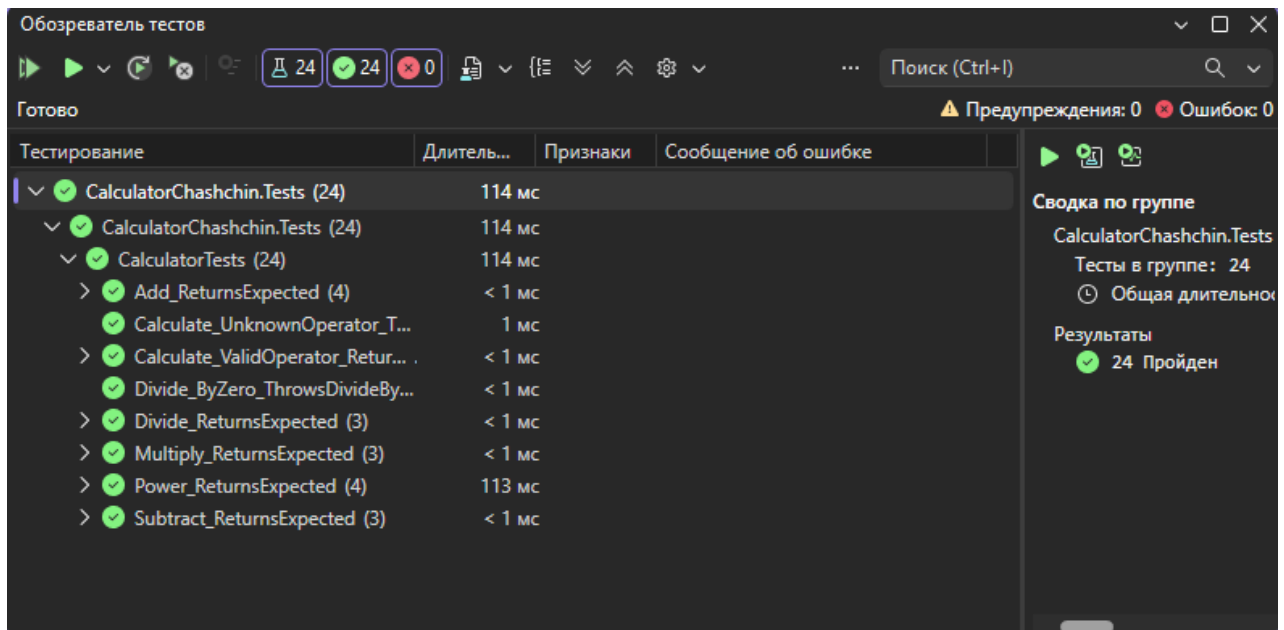
        [Fact]
        Ссылка 0
        public void Divide_ByZero_ThrowsDivideByZeroException()
        {
            Assert.Throws<DivideByZeroException>(() => _calc.Divide(5, 0));
        }

        [Theory]
        [InlineData(2, 3, 8)]
        [InlineData(9, 0.5, 3)]
        [InlineData(5, 0, 1)]
        [InlineData(2, -2, 0.25)]
        Ссылка 0
        public void Power_ReturnsExpected(double a, double b, double expected)
        {
            Assert.Equal(expected, _calc.Power(a, b), 10);
        }

        [Theory]
        [InlineData(2, 3, "+", 5)]
        [InlineData(10, 4, "-", 6)]
        [InlineData(6, 7, "*", 42)]
        [InlineData(20, 5, "/", 4)]
        [InlineData(2, 10, "**", 1024)]
        Ссылка 0
        public void Calculate_ValidOperator_ReturnsExpected(double a, double b, string op, double expected)
        {
            Assert.Equal(expected, _calc.Calculate(a, b, op), 10);
        }

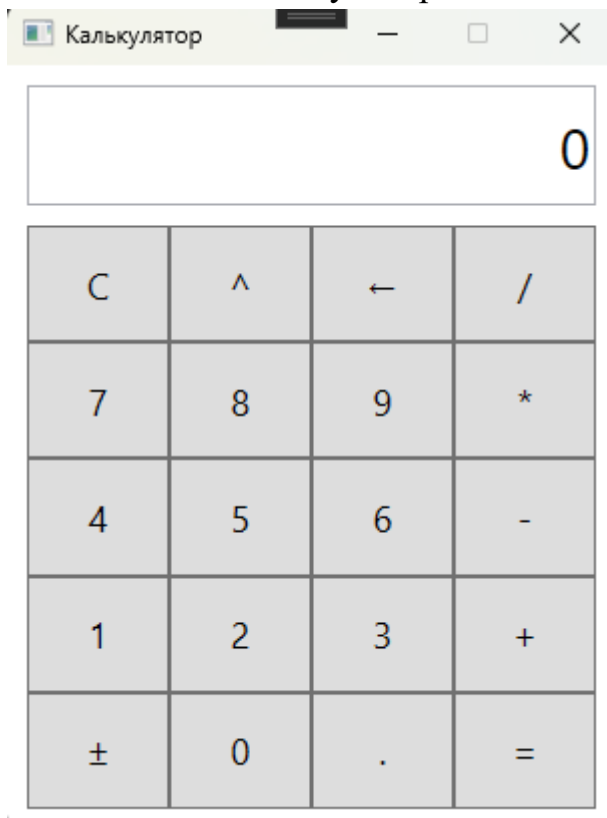
        [Fact]
        Ссылка 0
        public void Calculate_UnknownOperator_ThrowsArgumentException()
        {
            Assert.Throws<ArgumentException>(() => _calc.Calculate(1, 1, "%"));
        }
    }
}
```

6.3. Запуск тестов

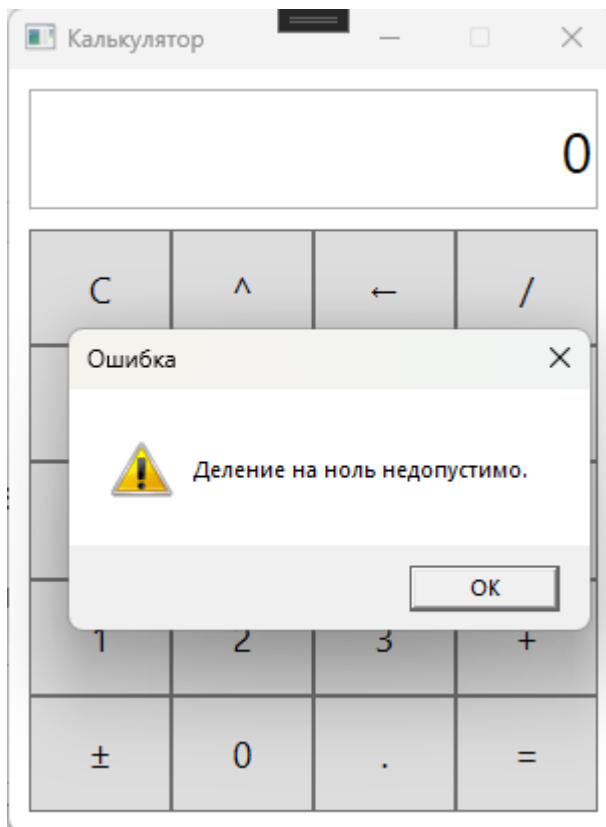


7. Запуск и проверка работы приложения

Итоговый вид калькулятора:



Ошибка при делении на 0:



8. Заключение

В ходе работы было разработано приложение «**Калькулятор**», реализующее арифметические операции $+$, $-$, $*$, $/$ и $^$ (возведение в степень).

Выполнены все поставленные задачи:

- Создано решение CalculatorChashchin в Microsoft Visual Studio 2022.
- Реализована библиотека классов CalculatorChashchin с пятью операциями.
- Разработан WPF-интерфейс CalculatorChashchin.WPF с кнопками цифр, операций, сброса, удаления, смены знака и десятичной точки.
- Обработаны исключительные ситуации (деление на ноль, неизвестный оператор).
- Написано 24 модульных теста на xUnit; все тесты успешно пройдены.
- Проведено ручное тестирование приложения.

Разделение логики и интерфейса на отдельные проекты делает решение удобным для сопровождения, повторного использования и тестирования, что соответствует современным подходам к разработке программного обеспечения.